SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	Beffy A Shanika	**Reg. No.:**	192511387
Date:	26/10/2026	**Duration:**	60 minutes

Code of Conduct

I Beffy A Shanika (192511387) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: beffy

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

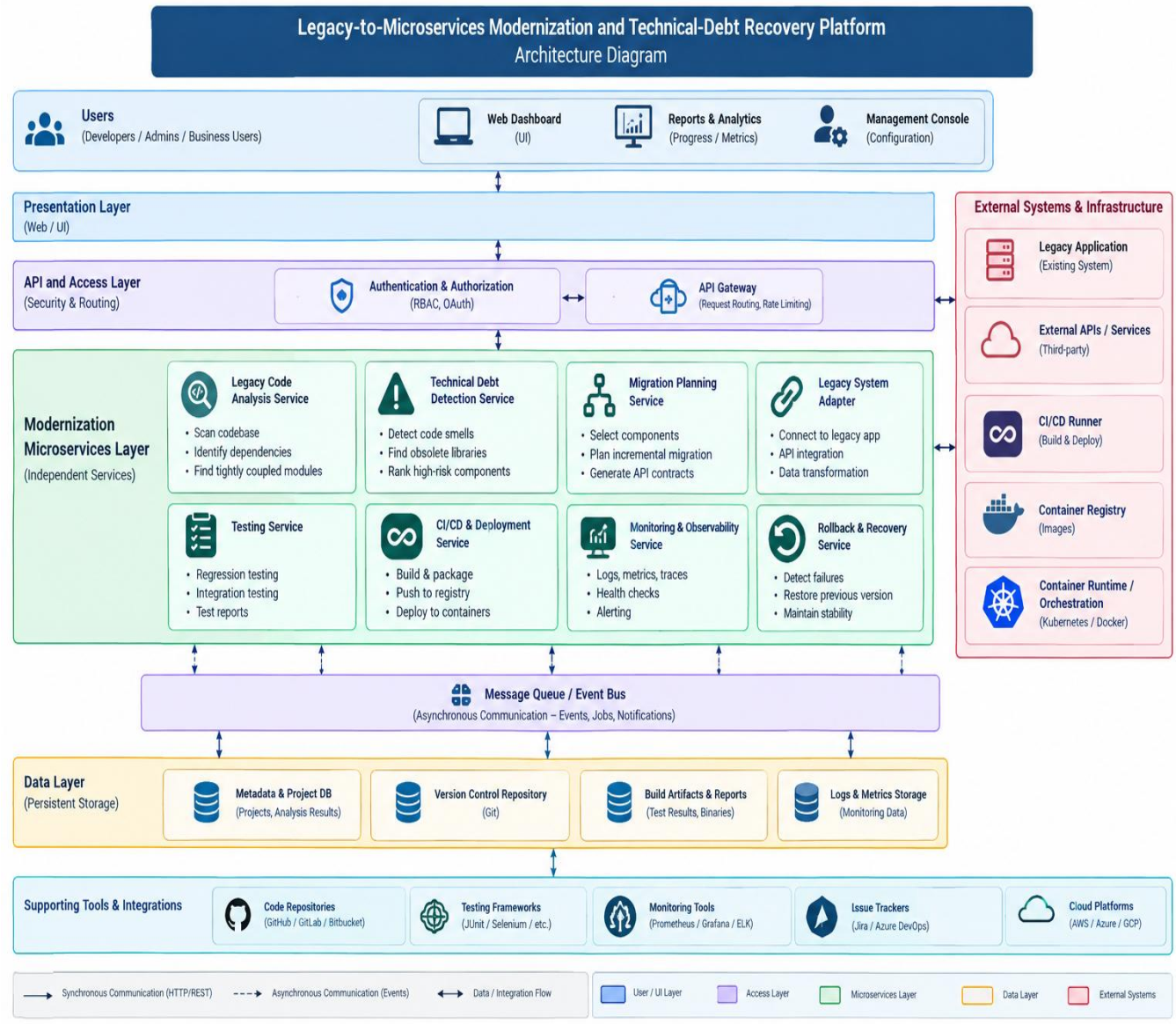
- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Legacy-to-Microservices Modernization and Technical-Debt Recovery Platform

Architecture Design



1. Analyze System Requirements

1.1 Introduction

Many organizations use legacy software systems that are difficult to maintain, modify, test, and scale. These systems may contain outdated libraries, tightly connected modules, and complex code. Completely replacing them can be expensive and risky because important business operations may be interrupted.

The proposed Legacy-to-Microservices Modernization and Technical-Debt Recovery Platform helps organizations gradually modernize legacy applications. It analyzes existing code, identifies technical debt, separates suitable modules into microservices, performs automated testing, and monitors the migrated services. It also supports version control, continuous integration and continuous deployment (CI/CD), and rollback to a previous stable version if a failure occurs.

1.2 System Objectives

The main objectives of the system are:

1. To analyze legacy applications and identify technical debt.
2. To detect tightly coupled modules, obsolete dependencies, and high-risk components.
3. To identify suitable modules for incremental migration.
4. To support API-based communication between legacy applications and new microservices.
5. To automate regression testing and integration testing.
6. To deploy migrated services using containers and CI/CD pipelines.
7. To monitor performance, availability, and migration-related failures.
8. To support rollback and provide reports on modernization progress.

1.3 Functional Requirements

Functional requirements describe the operations the system must perform.

ID	Functional requirement	Description
FR1	Application Analysis	Scan source code and identify dependencies and tightly coupled modules.
FR2	Technical Debt Detection	Identify code duplication, outdated libraries, complex code, and potential risks.
FR3	Migration Planning	Recommend suitable components for incremental migration.
FR4	API Management	Provide APIs for communication between legacy systems and migrated services.
FR5	Automated Testing	Execute regression and integration tests before deployment.
FR6	Containerization	Package migrated services into containers.
FR7	CI/CD Deployment	Build, test, and deploy services automatically.
FR8	Monitoring	Track errors, response time, resource usage, and service health.
FR9	Rollback	Restore the previous stable version when deployment fails.
FR10	Reporting	Generate reports showing technical debt, test results, and modernization progress.

1.4 Non-Functional Requirements

Non-functional requirements define how effectively and securely the system should operate.

Requirement	Description
Scalability	Support increasing workloads by scaling individual services.
Security	Protect source code, credentials, APIs, and application data.
Performance	Analyze code and process requests within acceptable time limits.
Reliability	Ensure migrated services work correctly and recover from failures.
Maintainability	Allow individual components to be updated without changing the entire platform.
Availability	Keep essential application functions accessible during migration and deployment.
Compatibility	Support different programming languages, databases, and legacy systems.
Testability	Verify functionality through automated regression and integration tests.

1.5 Key Quality Attributes

The most important quality attributes are scalability, maintainability, reliability, availability, performance, and security. These attributes influence the selection of a microservices-based architecture with supporting CI/CD, monitoring, and rollback mechanisms.

2. Select a Suitable Software Architecture

2.1 Selected Architecture: Hybrid Microservices Architecture

The proposed system uses a Hybrid Microservices Architecture. Microservices architecture is the primary style, supported by layered organization, API gateways, event-driven communication, and automated deployment pipelines.

In this architecture, the platform is divided into independent services. Each service performs a specific responsibility, such as code analysis, technical debt detection, migration planning, testing, deployment, or monitoring.

A layered structure organizes the user interface, API access, business logic, and data storage. Event-driven communication allows long-running analysis and testing tasks to execute asynchronously, while CI/CD automates building, testing, and deployment.

2.2 Why This Architecture Is Suitable

- Independent services: Code analysis, testing, monitoring, and deployment can be developed and maintained separately.
- Incremental modernization: Legacy modules can be migrated one at a time rather than replacing the entire system.

- Scalability: Resource-intensive services, such as code analysis and testing, can be scaled independently.
- Fault isolation: A failure in one supporting service can be isolated, with retries or recovery mechanisms where appropriate.
- Technology flexibility: Different services can use suitable programming languages and tools.
- Automation: CI/CD pipelines automate builds, tests, and deployments.
- Integration: APIs and adapters connect legacy applications with newly migrated services.

The architecture directly supports the project's main goal: modernizing legacy applications gradually while reducing migration risks and maintaining business continuity.

3. Software Architecture Diagram

The following diagram illustrates the overall architecture of the proposed platform, including users, application layers, modernization services, databases, external tools, and communication mechanisms.

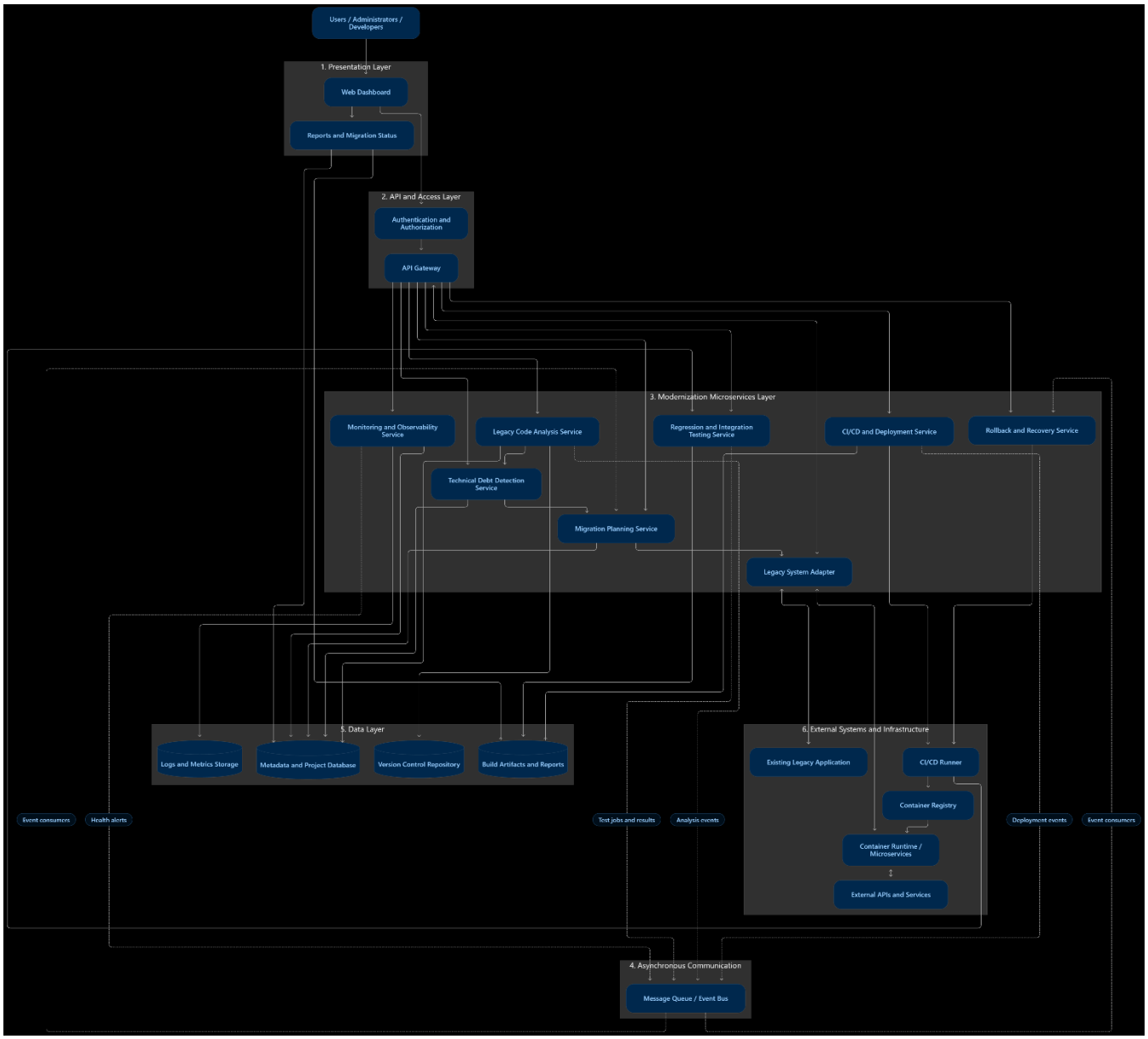


Diagram notation: solid arrows represent direct requests or data flows; dotted arrows represent asynchronous events. The diagram is a proposed logical design, not a claim that the platform has already been implemented.

Explanation of the Architecture Diagram

The system is organized into six main parts:

1. Presentation Layer: Provides the web dashboard for developers and administrators to upload or connect a legacy project, view analysis results, and track migration progress.
2. API and Access Layer: Authenticates users, checks permissions, and routes requests to the appropriate services.
3. Modernization Microservices Layer: Performs code analysis, technical debt detection, migration planning, testing, deployment, monitoring, and rollback.
4. Asynchronous Communication: Uses a message queue or event bus to process lengthy tasks without keeping the user waiting for every operation to finish.
5. Data Layer: Stores project metadata, source code references, reports, build artifacts, logs, and performance metrics.
6. External Systems and Infrastructure: Connects the platform to existing applications, external APIs, CI/CD runners, container registries, and the runtime environment for migrated services.

4. Explain Components and Their Interactions

4.1 Components and Their Responsibilities

Component	Purpose and responsibility
Web Dashboard	Allows users to submit projects, view technical debt, approve migration plans, and monitor results.
Authentication and Authorization	Verifies user identity and restricts access based on roles and permissions.
API Gateway	Routes incoming requests to the appropriate microservice and applies common access policies.
Legacy Code Analysis Service	Examines source code, dependencies, module relationships, and complexity.
Technical Debt Detection Service	Identifies outdated dependencies, duplicated code, code smells, and risky components.
Migration Planning Service	Uses analysis results to prioritize modules and plan incremental migration.
Legacy System Adapter	Connects existing application functions with new services through APIs or suitable adapters.
Testing Service	Runs regression and integration tests to check that migrated functionality remains correct.

CI/CD and Deployment Service	Automates building, testing, container packaging, and deployment.
Monitoring Service	Collects logs, metrics, errors, response times, and service health information.
Rollback and Recovery Service	Initiates recovery to a previous stable release when deployment checks fail.
Databases and Repositories	Store project metadata, version history, reports, logs, and artifacts.
Message Queue / Event Bus	Transfers background jobs and events between services asynchronously.

4.2 How Components Communicate

The architecture uses three main communication mechanisms.

1. Synchronous communication: REST APIs or HTTP requests are used when a component needs an immediate response. For example, the dashboard sends a request to the API gateway to retrieve a project's technical debt report.

2. Asynchronous communication: A message queue or event bus is used for long-running operations such as code scanning, testing, and deployment. A service publishes an event, and another service processes it when ready.

3. Database and repository access: Services store and retrieve the information required for their responsibilities. Version control tracks code changes, while databases store metadata and monitoring information. Each microservice should own its operational data where practical, rather than depending on unrestricted access to a shared database.

4.3 Overall Workflow of the System

Step 1: Submit the legacy application

The developer connects a source-code repository or submits an authorized project for analysis.

Step 2: Analyze the application

The analysis and technical debt services identify dependencies, complex modules, outdated libraries, and high-risk components.

Step 3: Prepare the migration plan

The platform recommends which components should be migrated first and identifies their dependencies and API requirements. A developer reviews and approves the plan.

Step 4: Refactor and develop services

Developers extract a suitable component into a microservice and connect it to the legacy application through APIs or adapters.

Step 5: Test the migrated service

Automated regression and integration tests verify that existing business functions continue to work correctly.

Step 6: Deploy the service

The CI/CD pipeline builds the application, packages it into a container, and deploys it after the required checks pass.

Step 7: Monitor and recover

The monitoring service tracks errors, availability, and performance. If deployment health checks fail, the rollback mechanism restores the previous stable version when safe to do so.

5. Discuss Quality Attributes

The proposed hybrid microservices architecture supports the following quality attributes.

Quality attribute	**How the architecture addresses it**
Scalability	Individual services such as code analysis and testing can be scaled independently according to workload.
Security	Authentication, role-based access control, secure APIs, encrypted communication, and secret management protect the platform.
Performance	Asynchronous processing prevents long-running analysis and testing tasks from blocking dashboard requests. Caching and optimized queries can improve response times.
Reliability	Automated tests, health checks, retries, fault isolation, and rollback mechanisms reduce the impact of failures.
Maintainability	Each microservice has a defined responsibility, making it easier to modify, test, and update components independently.
Availability	Redundant service instances, health monitoring, and controlled deployments help keep essential functions available during updates.

5.1 Evaluation of Quality Attributes

- Scalability: The platform can handle larger projects by allocating more resources to the services that need them.
- Security: Only authorized developers can access projects and trigger migrations. Source code and credentials must be protected throughout the pipeline.
- Performance: Background tasks run asynchronously, while frequently requested reports can be cached where appropriate.
- Reliability: Regression tests verify existing business behavior, and deployment failures can trigger a controlled rollback.
- Maintainability: Independent services reduce the need to change the entire platform when one function is modified.
- Availability: Rolling or canary deployments can reduce downtime. High availability still requires redundancy, proper configuration, and recovery planning.

These quality attributes should be verified using measurable indicators such as API response time, service uptime, test pass rate, deployment failure rate, and recovery time.

6. Justify Architectural Decisions

6.1 Rationale for the Selected Architecture

The hybrid microservices architecture is selected because the project itself focuses on converting tightly coupled legacy applications into independently manageable services.

The architecture supports incremental migration, automated testing, containerized deployment, and continuous monitoring. Its modular structure also makes it easier to add new analysis tools and integrate with different legacy technologies.

The supporting architectural patterns serve specific purposes:

1. Layered architecture: Separates the user interface, API access, business logic, and data management.
2. Microservices architecture: Divides modernization functions into independently maintainable services.
3. Event-driven architecture: Supports asynchronous processing of analysis jobs, test results, and deployment notifications.
4. API-first design: Enables communication between legacy applications and migrated services.
5. CI/CD and containerization: Make deployments repeatable and support consistent testing and execution environments.

6.2 Trade-Offs Considered

Advantages	Disadvantages
Individual services can scale independently.	Requires additional infrastructure and configuration.
Supports gradual migration instead of a complete replacement.	Managing communication between services is more complex.

Allows independent development and deployment.	Distributed debugging and testing can be difficult.
Supports automated deployment and rollback.	CI/CD pipelines require careful setup and maintenance.
Allows integration with different technologies.	API compatibility and data consistency require attention.

Although microservices introduce additional complexity, this trade-off is justified because the platform is specifically designed to manage modernization of complex legacy applications.

6.3 Support for Future Enhancements

The architecture can be extended in the following ways:

- AI-assisted code analysis: Add services that recommend refactoring opportunities and migration strategies.
- Additional language support: Integrate analyzers for Java, Python, C#, and other programming languages.
- Cloud integration: Support deployments across different cloud platforms and container orchestration systems.
- Advanced monitoring: Add distributed tracing, anomaly detection, and predictive failure analysis.
- Security scanning: Integrate automated vulnerability scanning and dependency auditing.
- Migration analytics: Compare maintenance effort, deployment frequency, failure rates, and response times before and after modernization.
- Additional integrations: Connect with issue trackers, code repositories, testing frameworks, and enterprise monitoring tools.

These enhancements can be introduced as new services or integrations without requiring a complete redesign of the platform.

7. Conclusion

The Legacy-to-Microservices Modernization and Technical-Debt Recovery Platform uses a hybrid microservices architecture to support the gradual transformation of legacy applications into modern, independently manageable services.

The architecture combines layered organization, API-based communication, event-driven processing, automated testing, containerization, CI/CD, monitoring, and rollback mechanisms. These components help identify technical debt, reduce migration risks, and maintain existing business functionality during modernization.

The proposed design improves the potential for scalability, maintainability, reliability, security, and availability. Its effectiveness can be evaluated through measurable indicators, including technical debt reduction, regression test results, deployment success rate, service availability, and performance improvements.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	Mark
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25